

```
$ msginit -l te_IN -o te.po -i hello18n.pot
```

This program will ask for the email of the user (given as `x@y.com` in this example), and use it as the contact address in the header. Now open the file in an editor (for example, *gedit*), and type in the translation for the string `"Hello World\n"` as `"సమస్యలను\n"` in the target language Telugu, and save the file. The resultant file is shown in Listing 5.

Listing 5: te.po corresponding to hello18n.pot

```
# Telugu translations for hello18n package.
# Copyright (C) 2012 THE hello18n'S COPYRIGHT HOLDER
# This file is distributed under the same license as the hello18n
package.
# arjun <x@y.com>, 2012.
#
msgid ""
msgstr ""
"Project-Id-Version: hello18n 1.0\n"
"Report-Msgid-Bugs-To: \n"
"POT-Creation-Date: 2012-06-04 22:43+0530\n"
"PO-Revision-Date: 2012-06-04 22:54+0530\n"
"Last-Translator: arjun <x@y.com>\n"
"Language-Team: Telugu\n"
"Language: te\n"
"MIME-Version: 1.0\n"
"Content-Type: text/plain; charset=UTF-8\n"
"Content-Transfer-Encoding: 8bit\n"

#: hello18n.c:15
#, c-format
msgid "Hello World\n"
msgstr "సమస్యలను\n"
```

Let's use the `po` file to build machine object (`.mo`) version, which is also called as message catalogue and store it in a specified location, with the filename changed to the package name. To do this, you may need superuser access as the standard location for storing such files has restricted access. If you do not have this, you can use your home directory, with appropriate replacements in the `bindtextdomain` function call of source code. The message catalogues are placed in a directory structure consisting of *language code/LC_MESSAGES* in the designated path. For example, Telugu messages are placed in `/usr/share/locale/te/LC_MESSAGES/hello18n.mo` and Hindi messages are placed in `/usr/share/locale/hi/LC_MESSAGES/hello18n.mo`.

This allows supporting additional locales by placing the localised message catalog files, without the need for recompiling the source, as the run-time library will automatically check for the translated message catalogues, and if it does not find any, will use the default messages. This also provides flexibility for the translation to progress independently.

```
$ msgfmt -o te.mo -v te.po
```

```
$ cp te.mo /usr/share/locale/te/LC_MESSAGES/hello18n.mo
```

Results

Let's run the program with the default English locale, and then change the locale to Telugu. After which, let's run the program again, and then change the locale back to English. Note that the Telugu text rendering will not be perfect in a typical terminal, due to limitations in rendering the complex script of the language. You can direct the output to a file, and view the properly rendered message by opening that file in an editor. You can also use Terminator software (from software.jessies.org) which can display complex scripts properly. If you are using an older system, you can use *LANG* in place of *LANGUAGE* for the proper operation of the following code. Alternatively, you can change your language settings to the desired language, log out, and log in with the new set-up, and try the code.

```
$ echo $LANGUAGE
en_US.utf8
$ ./hello18n
Hello World
$ export LANGUAGE=te_IN:te
$ ./hello18n
సమస్యలను
$ export LANGUAGE=en_IN:en
```

That concludes our primer on Gettext. We looked at how a typical 'C' language source can be instrumented to support internationalisation. We also looked at how the messages are extracted using *xgettext*, translated using *msginit*, and placed in the proper location, so that the run-time library can use them.

Several tool-kits, Web-based tools and standalone software were developed to allow the localiser to focus on translation without worrying about the nitty-gritty of files and formats. We will cover a few popular tools in forthcoming articles. 

Resources

- Introduction to Internationalisation Programming, Issue 103, November 2002, By Oleksy Ye Tykhomyrov in the Linux Journal, <http://www.linuxjournal.com/article/6176?page=0,0>
- Gettext manual: <http://www.gnu.org/software/gettext/manual/gettext.html>
- Terminator Cross platform GPL Terminal emulator with complex text rendering support. <http://software.jessies.org/terminator/>

By: Arjuna Rao Chavala

The author is an independent consultant in the areas of IT, program/engineering management and open source. He is also the president of Wikimedia India and the WG Chair for IEEE-SA project P1908.1 "Virtual keyboard standard for Indic languages". He can be reached through his website <http://arjunarao.blogspot.in> and Twitter id @arjunarao.